Viva Questions & Complete Answers

Multi-Agent Financial Research System -- 45 probable questions with model answers

A. PROJECT BASICS

Q1. What is the main objective of your project?

To turn 100+ page financial filings into trustworthy research in seconds. The system uploads an annual or quarterly report, extracts financial metrics, flags risks, answers questions with citations, and generates reports and benchmarks -- with every number traceable back to the exact source line in the filing.

Q2. What problem does your project solve?

Three problems: manual analysis of long reports takes hours; AI chatbots invent numbers and show no sources; and small teams cannot afford expensive terminals. Our system is fast, evidence-based, and runs at zero infrastructure cost.

Q3. Why did you choose this project?

It sits at the intersection of AI and finance -- a real, measurable pain point. The interesting engineering challenge is not 'make an AI talk about finance', it is 'make AI that cannot lie about numbers'. That verification-first design made this project technically meaningful.

Q4. Who are the target users of this system?

Financial and equity analysts, students and researchers, individual investors, small businesses and startups without analyst teams, investor-relations and audit support teams, and educators teaching financial analysis.

Q5. What are the key features of your project?

Upload-time document validation; automatic chunking and indexing; extraction of financial metrics with stored evidence lines; red-flag risk detection with exact quotes; research Q&A with [S1]-style citations; structured report generation; company-vs-company benchmarking; optional semantic search; a full agent activity trail; and graceful degradation when AI is unavailable.

Q6. What makes your solution different from a traditional AI chatbot?

A chatbot answers from memory -- we answer only from YOUR uploaded documents. Every claim carries a clickable citation; every metric stores its source line; numbers are computed by code, not predicted by an AI; and an honesty layer drops any claim it cannot verify. The output is structured research -- tables, flags, reports -- not chat text.

B. MULTI-AGENT ARCHITECTURE

Q7. What is a multi-agent AI system?

A system of specialised agents where each agent handles one sub-task -- validation, extraction, risk scanning, retrieval, writing, comparing -- coordinated by an orchestrator, sharing data through a common database, each with its own prompts, fallbacks and tests.

Questions & Answers (continued)

Q8. Why did you choose a multi-agent architecture instead of a single AI agent?

Isolation, specialisation and trust. One big prompt is a black box: when it fails, everything fails. Separate agents let deterministic code handle numbers while AI handles language, one failing stage cannot kill the pipeline, and every agent is independently testable and auditable.

Q9. How many agents are involved in your system?

Eight specialised agents plus one orchestrator: Validation, Document, Extraction, Red Flag, Embedding, Research, Report and Benchmark Agents, all coordinated by the Pipeline Orchestrator.

Q10. What is the role of each agent?

Validation: confirms the upload is really a financial document and rejects wrong types by name. Document: chunks and indexes text. Extraction: pulls financial metrics with evidence. Red Flag: finds risks with exact quotes. Embedding: builds semantic vectors (optional). Research: answers questions with citations. Report: writes the structured analysis report. Benchmark: compares companies safely.

Q11. How do the agents communicate with each other?

Loosely, through shared PostgreSQL tables -- documents, chunks, metrics, risk flags, reports and the agent_logs activity table. The orchestrator sequences the automatic pipeline; on-demand agents simply read what the pipeline stored. This loose coupling means any agent can fail, restart, or be replaced without breaking the others.

Q12. What is the role of the orchestrator agent?

The Pipeline Orchestrator (src/lib/agents/orchestrator.ts) claims queued jobs atomically -- so two workers can never run the same job -- then runs the stages Document -> Extraction -> Red Flag -> Embedding in order, isolates each stage with its own error handling, retries failures, and reclaims crashed jobs via a 15-minute stale-lock timeout.

Q13. How do you manage dependencies between agents?

Dependencies are data dependencies: extraction needs chunks, risk analysis uses text and metrics, embeddings need chunks. The orchestrator runs stages in dependency order and each stage writes the data the next stage reads. Job statuses (queued / completed / retrying / failed / partial) record exactly how far each document got.

Q14. Can multiple agents work in parallel?

Yes across documents and requests: the atomic job claim lets several workers process different documents at the same time safely, and research / report / benchmark requests run concurrently on demand. Within one document the stages run in order by design, because each stage depends on the previous stage's data.

Q15. What happens if one of the agents fails?

The failure is isolated: the stage is marked failed or partial in agent_logs, the remaining stages still run, and the job retries automatically up to its attempt limit. Each agent also has a fallback -- deterministic extraction when AI fails, keyword search when embeddings are off -- so the user still gets a usable result and an honest status.

C. TECHNOLOGY & DATA

Questions & Answers (continued)

Q16. Which technologies and frameworks did you use?

Next.js 16 (App Router) with TypeScript and Tailwind CSS for the application; PostgreSQL (Neon) with Drizzle ORM for storage; NextAuth with bcrypt for authentication; Groq's OpenAI-compatible API for the LLM; optional embeddings via any OpenAI-compatible provider (Ollama locally, Gemini or Nomic in production); pdf-parse for document extraction; deployed on Render with Node 22.

Q17. Which LLM are you using, and why?

Groq-hosted open models (Llama / GPT-OSS class). Reasons: very fast inference for interactive research, a generous free tier that fits our zero-cost goal, an OpenAI-compatible API so the provider can be swapped without code changes, and reliable structured-JSON output -- which our validation layer depends on.

Q18. How do you collect financial and business data?

The user uploads the primary source documents -- annual reports, quarterly filings, earnings PDFs or TXT files. The system parses the text, chunks it, and extracts every figure from the filing itself. We deliberately do not scrape the web: the trust model starts from documents the user can see and verify.

Q19. What data sources does your system use?

User-uploaded financial documents (annual and quarterly reports, 10-K/10-Q style filings). Our demo dataset contains 28 PDFs -- seven companies across FY2023-2026 including Apple, Microsoft, Amazon, Google, Meta, NVIDIA and Infosys. All extracted data lives in 12 PostgreSQL tables.

Q20. How do you ensure that the collected data is reliable?

Layer by layer: the Validation Agent confirms the file really is a financial document; every extracted metric stores the exact evidence line it came from; a deterministic parser re-computes values and overrides the AI whenever they conflict; a metric is saved only if its evidence supports the number; and every risk flag carries a verbatim quote.

D. ANALYSIS, INSIGHTS & TRUST

Q21. How does the system perform financial analysis?

The Extraction Agent builds the metric set -- revenue, growth, profitability, margins, per-share, balance-sheet and cash-flow figures. Ratios and growth rates are computed by code from stored numbers. The Red Flag Agent adds computed trend risks (leverage, liquidity, margin pressure) and filing-language risks, giving a full analytical picture.

Q22. How does the system generate business insights?

The Report Agent writes an executive summary, key-financials comparison table, risk analysis and analyst recommendations strictly from the stored metrics and risk flags. The Benchmark Agent produces verified comparisons. Insights are narrative written by AI, but the facts underneath are always code-verified data.

Q23. Can the system analyze competitors and market trends?

Yes for competitors from filings: the Benchmark Agent compares two companies side by side on the same fiscal basis -- revenue, margins, growth, leverage, cash flow -- and refuses mismatched periods. Trends are analysed across fiscal years from extracted metrics. Live stock prices and news are intentionally out of scope -- a stated limitation.

Questions & Answers (continued)

Q24. How does the system identify financial risks?

Two detectors working together: textual red flags -- going-concern language, auditor qualifications, debt and litigation phrases -- matched from filing language with severity levels and exact quotes; and financial trend risks computed from the metrics, such as high debt-to-equity, weak current ratio, or cash-flow deterioration.

Q25. How do you prevent hallucinations in AI-generated reports?

Six independent guards: answers come only from retrieved source text; every claim must include an exact supporting quote; numeric tokens in a claim must appear in that quote or the claim is dropped; metric values are computed by code and override the AI; benchmark prompts receive exact packed values the AI cannot alter; and unanswerable questions get an honest refusal instead of an invented answer.

Q26. How do you validate the accuracy of the generated insights?

At runtime: citation validation, evidence-line checks and the local-metric override run on every request. At development time: 55 automated unit tests cover extraction, grounding, AI fallbacks and parsing; the Validation Agent passed a 10-document acceptance matrix; and the agent_logs trail lets any output be audited after the fact.

Q27. How do you handle missing or inconsistent data?

Honestly. Missing metrics are stored as null and displayed as N/A; a metric without supporting evidence is not saved at all; incompatible fiscal periods raise a clear warning instead of a misleading comparison; and when the deterministic parser and the AI disagree, the code-computed value wins and the conflict is recorded.

E. EVALUATION, PERFORMANCE & SECURITY

Q28. How do you evaluate the performance of individual agents?

Each agent has a testable function surface and unit tests for its logic; every run writes a row to agent_logs with agent name, action, status and duration; and targeted acceptance suites exist -- for example the Validation Agent's accept/reject matrix (real annual reports accepted; resumes, invoices, novels rejected by name).

Q29. What metrics do you use to evaluate the overall system?

Extraction correctness (against source lines), citation accuracy (claims vs quotes), risk coverage, report completeness (required sections present), per-stage latency, fallback activation rate, and end-to-end task success: upload -> question -> verified answer. User-facing trust is measured by verifiability: can any answer be audited to the filing?

Q30. How do you optimize the system's response time and cost?

Fast free-tier Groq inference; small targeted prompts in JSON mode; retrieval limited to the top-8 chunks with 1500-character chunks; extracted data cached in PostgreSQL so reports and benchmarks never re-parse documents; batch job limits; retry backoff instead of wasteful re-asks; and optional AI -- the deterministic fallbacks keep the system usable at zero token cost.

Q31. How do you ensure data privacy and security?

Passwords are bcrypt-hashed and sessions are managed by NextAuth; every database query is scoped to the logged-in user; background workers require a CRON_SECRET and seeding requires a SEED_SECRET; uploads have size caps and content validation; and document text is only sent to the AI provider you explicitly configure -- including local-only Ollama.

Questions & Answers (continued)

Q32. Can the system handle large volumes of financial data?

Yes at team scale by design: documents are chunked and indexed once, jobs are queued and claimed atomically so multiple workers can process documents in parallel, and all analysis reads pre-stored data rather than re-parsing. For enterprise volume the same design scales with managed Postgres, a vector index and a worker fleet.

F. REPORTS, CUSTOMISATION & LIMITATIONS

Q33. How does the system generate the final research report?

The Report Agent reads the stored companies, metrics and risk flags; builds the key-financials comparison table directly in code; asks the LLM to write the executive summary, risk analysis and recommendations strictly from that data; validates the output; and saves it to the analysis_reports table -- viewable in the app and downloadable as a text report.

Q34. Can users customize the analysis based on their requirements?

Yes: users choose the company, ticker, document type and fiscal year at upload; organise documents into research sessions; ask their own free-form questions with citations; pick any two companies for benchmarking; and reprocess a document when they upload a corrected file.

Q35. What are the limitations of your current implementation?

No OCR for scanned image PDFs; no live market prices or news data; report export is text/Markdown rather than PDF; semantic search needs an embeddings provider (otherwise keyword search is used); free-tier hosting means cold starts and a 4.5 MB upload cap; and benchmarks require compatible fiscal periods by design.

Q36. What challenges did you face during development?

PDF extraction across varied and corrupted filing formats; designing citation validation strict enough to block hallucinations but useful enough to keep good answers; surviving AI-provider failures with graceful degradation instead of errors; retrieval quality -- filler words and finance synonyms initially drowned the signal; and safe concurrent job processing with atomic claims and stale-lock recovery.

Q37. What are the potential real-world applications of this project?

Equity and credit research desks, investor-relations self-service, audit and due-diligence support, portfolio screening, academic financial analysis, fintech advisory on-ramps, and any team that must read long filings quickly and prove where every number came from.

G. STRATEGY & FUTURE

Q38. How can this system help business leaders make better decisions?

It compresses hours of reading into minutes of cited insight, surfaces risk language early with exact quotes, makes comparisons consistent and period-safe, and -- crucially -- lets decision-makers verify every figure, so confidence is based on evidence rather than trust in a black box.

Questions & Answers (continued)

Q39. Can this system be integrated with existing financial platforms?

Yes. The application is API-first -- 20 REST endpoints for documents, research, reports, benchmarks and health; standard PostgreSQL storage and NextAuth identity; and an OpenAI-compatible AI layer. Existing portals can call our APIs or read our tables directly.

Q40. How would you scale this solution for enterprise use?

Managed Postgres with pgvector for hybrid retrieval, a fleet of stateless workers (the atomic job claim already supports this), dedicated LLM and embedding endpoints or on-premises models for compliance, caching and CDN for the UI, RBAC and SSO on top of the existing per-user scoping, and monitoring built on agent_logs.

Q41. What happens if two agents produce conflicting results?

Deterministic code wins: if the AI's extracted value conflicts with the local parser, mergeWithLocalMetrics keeps the code value; claims without verifiable quotes are dropped; benchmark guards block invalid period comparisons. Conflicts are recorded in agent_logs so the disagreement itself is auditable.

Q42. How do you manage token usage and LLM costs?

Small prompt templates with strict JSON schemas; only the top-8 retrieved chunks and 800-character excerpts enter the prompt; conversation history is capped; failed calls use backoff retries instead of hammering the API; extracted data is reused from the database; and Groq's free tier keeps monetary cost at zero.

Q43. What is the difference between an AI agent and a traditional workflow?

A traditional workflow executes fixed predefined steps regardless of content. An AI agent perceives its context -- here, the retrieved document evidence -- reasons about the goal, chooses what to do, and verifies its output against sources, while still being orchestrated with deterministic safety rails. Our system is deliberately both: agentic where judgement is needed, workflow where reliability is needed.

Q44. Why is agentic AI suitable for financial research?

Filings are long, heterogeneous and multi-step by nature: validate, parse, extract, flag, retrieve, compare, write. Each step needs a different technique -- regex for numbers, LLM for language -- and every step needs verification. The agentic decomposition gives exactly that: specialisation, evidence at each step, and failure isolation.

Q45. What future enhancements would you introduce to this project?

PDF and Excel report export; OCR for scanned filings; pgvector-based hybrid semantic search; live market data and news sentiment; valuation models (DCF, comparable multiples); streaming answers; multi-user teams with role-based access; a formal RAG evaluation harness; and multi-language support.

Golden line for any hard question: 'Numbers by code, language by AI, trust by evidence.'